



TECHNICAL SPECIFICATION

Development Hardware

Capacity model & build tiers — salvage to primary

PREPARED FOR

Infrastructure Team

Procurement, assembly & triage

Tier	0 salvage · 1 upgraded · 2 mid · 3 primary
Cost range	0 – 34.8 million IDR per box
Capacity unit	Seat & stack
Companion documents	Setup Guide · Developer Guide

Scope

This document converts hardware into capacity, then lists the build tiers that hit each capacity point, and ends each hardware class with what must be done to the machine **before the operating system boots for the first time**. It says nothing about what runs on the box or how work is organised: the companion **Setup Guide** provisions every tier identically from that first boot onward, and the **Developer Guide** describes a workflow that is the same on every tier. The only thing a tier changes is how many people and how many stacks fit.

The hand-off point is the same for every class, and it is a state, not a feeling: the Setup Guide’s first command is `ansible-playbook` as **root** over the LAN, so everything that command needs must already be true. That is Ubuntu Server on the SSD, wired ethernet, MemTest86 passed, the hostname already equal to the name the box will carry in the inventory, and the operator’s key on **root** — not merely on the account the installer created. [Installing the operating system](#) is the runbook for the last three, because the installer does not give you them and the gap is not obvious until a provisioning run stops on it.

From there the Setup Guide takes over and the hardware no longer matters — except for the few facts the runbooks below tell you to declare in the box’s inventory entry: laptop lid, fan control, a second disk, ethernet ports with nothing plugged into them, and whether a crash-dump kernel is wanted.

The document is written to survive hardware changes. The capacity model in the next section is the durable part: new machines are placed on its tables rather than described from scratch. The tier list is the perishable part — prices, part numbers, and what counts as “current” all move.

Capacity model

Sizing primitives

Every capacity number in this repository derives from the table below. Figures are resident set size under a Spring Boot + Postgres profile, rounded up. Measure again if the workload profile changes materially — a Node or Python service costs roughly half a JVM seat, an Oracle-backed stack rather more.

Item	Resident	Multiplicity
Fixed host overhead		
Ubuntu Server, headless (<code>multi-user.target</code>)	0,7 GB	per box
Rootless Docker daemon	0,2 GB	per active user
Registry pull-through mirror	0,2 GB	per box, if hosted
Per seat — one developer actively working		
Claude Code session	0,6–1,5 GB	per seat, grows with context
Running application JVM (512 MB heap)	1,2 GB	per seat
Build: Maven launcher + forked surefire JVM	2,0 GB	per seat, transient
Per stack — the services one project needs		
Minimal — app + Postgres	1,5 GB	per stack

Item	Resident	Multiplicity
Light — app + Postgres + Redis or one broker	3,0 GB	per stack
Full — apps + 3-broker Kafka + Oracle + monitoring	10–12 GB	per stack

A **seat** is one developer working: a Claude Code session, an application running, and a build that may fire at any moment. Peak and steady state differ by the build, and the build is what sizing must survive:

Seat state	Components	Total
Steady — app running, agent thinking	1,2 + 1,0	2,2 GB
Peak — build in flight alongside the app	1,2 + 2,0 + 1,0	4,2 GB
Peak, sequenced — build replaces the app	2,0 + 1,0	3,0 GB

Budget **5 GB per concurrent seat** against peak, plus that seat’s stack. A seat on a light stack therefore costs $5 + 3 = 8$ GB. *Sequenced* means the developer never builds while the application runs — a real discipline on small boxes, and the difference between one seat fitting and not fitting.

RAM tiers

RAM is the primary constraint on every box below the top tier. “Usable” is the total less the fixed host overhead and one rootless daemon per seat.

RAM	Usable	Seats	Full stacks	What this tier is for
8 GB	7,1 GB	1 sequenced	0	Standing E2E loop runner. One seat only if it never builds and runs at once. Not multiuser.
16 GB	14,9 GB	1 comfortable 2–3 light	1	One seat on the sizing profile above. Two or three seats only where the work is lighter than that profile — site maintenance, docs, interpreted runtimes.
32 GB	30,5 GB	2–3	2	The entry tier for real multiuser work.
64 GB	61,9 GB	5–7	4–5	Team box or heavy single-user stacks.
128 GB	125,7 GB	12+ (CPU-capped first)	6–8	Primary workstation. RAM stops being the binding constraint.

CPU tiers

Cores do not change how many stacks fit — they change how many **builds** can run at once before they start stealing time from each other. A build that slows under contention produces timing flakes, which is the failure mode this platform exists to eliminate, so this ceiling is treated as hard.

Class	Cores/threads	Concurrent builds	Note
Dual-core mobile (Broadwell-U)	2c/4t	1	Build and agent contend on the same two cores.
Quad mobile (Haswell-H)	4c/8t	1–2	Thermally limited before it is core-limited.

Class	Cores/threads	Concurrent builds	Note
Quad desktop, no HT (Haswell)	4c/4t	2	Higher sustained clock than its mobile counterpart.
Quad desktop, HT (Haswell)	4c/8t	2–3	
16-core desktop (Zen 5)	16c/32t	6–8	Homogeneous cores — no scheduling jitter.

Disk tiers

Disk is first a gate, then a throughput cap.

Media	Verdict	Concurrent builds	Note
Spinning HDD	Disqualified	0	Docker overlay churn and Maven writes make this unusable at any RAM.
SATA SSD	Minimum	2	≈550 MB/s. Adequate; page cache matters more here than on NVMe.
NVMe Gen3	Good	4	
NVMe Gen4, single	Good	6	
NVMe Gen4, split OS/ Docker	Full	8+	Overlay churn and volume I/O leave the OS disk entirely.

Space, per box:

Consumer	Budget
Ubuntu Server + toolchains	20 GB
Per user: rootless image store (no sharing between users)	25 GB
Per user: dependency caches (~/.m2, pnpm store, uv, Composer)	10 GB
Per user: worktrees and their build output	5 GB
Registry mirror data, if hosted	30 GB

Per-user image stores are the price of isolation. Rootless Docker gives each user their own image store, so a shared image is stored once per active user rather than once per box. The registry pull-through mirror makes this cost disk but not bandwidth. Budget 25 GB per user and it is a non-issue; forget it and a 128 GB SSD fills in a fortnight.

The binding constraint rule

A box's capacity is the **minimum** across the three tables, not the average and not the headline number:

$$\text{seats} = \min(\text{RAM seats}, \text{CPU concurrent builds} \times 1,5, \text{disk concurrent builds} \times 1,5)$$

The 1,5 multiplier on the build-derived ceilings reflects that seats do not all build simultaneously — a seat spends most of its time with the agent thinking or the developer reading, not compiling. It is an oversubscription factor, and it is the only one in this model.

A 32 GB box with a dual-core CPU is a dual-core box: three seats fit in memory, but the second concurrent build already contends, so it seats one and a half — call it one seat plus a standing loop. Adding RAM to that machine buys nothing. **Identify the binding constraint before spending money on any upgrade.**

Worked examples

Machine	RAM seats	CPU	Disk	Seats	Binding constraint
Salvaged laptop — 2c/4t, 8 GB, SATA SSD	1 seq.	1,5	3	1 sequenced	RAM and CPU together
Salvaged desktop — 4c/4t, 8 GB, SATA SSD	1 seq.	3	3	1 sequenced	RAM
Same desktop upgraded to 32 GB	2–3	3	3	2–3	None — balanced
Mid build — 8c/16t, 64 GB, NVMe	5–7	6	6	5–6	CPU
Primary — 16c/32t, 128 GB, 2×NVMe Gen4	12+	9–12	12+	6–8 full stacks	Stack size, not seats

The third row is the point of this whole model: the upgrade that moves a salvaged desktop from one sequenced seat to a genuine multiuser box is **RAM alone**, because RAM was the binding constraint and the CPU already had headroom. That upgrade is the cheapest capacity in this document.

What the tier writes into the box

A box's tier is a group in the Ansible inventory, and the group carries the values below. They are the seat budget made enforceable: **MemoryHigh** is where one user's work starts being throttled, **MemoryMax** where it is killed, the quota how much of the disk one home directory may hold. The Setup Guide explains the mechanism; this table is rendered from the same files the playbook reads, so it cannot disagree with what a box actually gets.

Tier	Group	Min RAM	MemoryHigh	MemoryMax	TasksMax	Quota soft / hard
0	Salvage tier0_salvage	7 GB	5G	6G	4096	40G / 45G
1	Compact tier1_compact	15 GB	2400M	2800M	4096	30G / 35G
2	Salvage, upgraded tier2_upgraded	30 GB	8G	10G	8192	60G / 70G
3	Mid-range build tier3_mid	62 GB	8G	10G	8192	100G / 120G
4	Primary workstation tier4_primary	125 GB	16G	20G	16384	250G / 300G

Two things take capacity before the tier ever sees it, both found commissioning the first box, and both worth checking against the numbers above rather than trusting the sticker:

- **kdump reserves memory at boot.** Ubuntu's default `crashkernel=` takes 512 MB on a machine in the 4–32 GB band, so a 16 GB box reports 15.405 MB to the tier check, not 15.917 MB. On a box that is reprovisioned from git rather than debugged from a crash dump, that is 3% of RAM bought for nothing. The Setup Guide removes it.
- **The installer allocates only part of the disk.** Guided LVM leaves the remainder of the volume group unassigned — 100 GB of a 220 GB SSD in use and the rest idle, on the first box. The disk budget below assumes the whole device, so extend the volume before counting seats.

Both are software, and so belong to the Setup Guide's first-boot steps. They appear here because their effect is on capacity. The kdump reservation is declared away per box with `dev_kdump_enabled: false`, so it is not a thing to remember twice.

The `MemoryMax` values very nearly sum to physical memory at each tier's seat count on purpose: a single runaway seat is killed by its own ceiling before it can drive the box into global OOM and take the other seats with it. Tier 1 reaches the same guarantee from a different workload: its ceilings are sized against a light seat rather than a JVM one, and its own section shows the arithmetic.

Tier 0 — Salvage

Existing hardware, zero purchase. The role is real but narrow: **standing E2E loop runner**, or one seat. Same platform, same workflow, one person at a time — a second seat does not fit in 8 GB, and pretending otherwise produces OOM kills that read as flaky tests. The developer on a Tier 0 box works *sequenced*: one stack at a time, and never a build while the application runs.

Triage gates

Apply in order. A machine that fails any gate is not a candidate; there is no partial credit and no configuration that compensates.

Gate	Requirement	Rejection reason
1	Storage is an SSD — SATA or NVMe	Docker overlay churn plus Maven writes on a spinning disk make the box unusable at any RAM. A cheap SATA SSD clears this gate; nothing else does.
2	64-bit CPU with 4 threads or more	Below this, the agent session and a build contend for the same two cores and every loop becomes serial.
3	8 GB RAM minimum, and the board takes more	8 GB is the floor for one sequenced seat. A board that cannot exceed it is a permanent Tier 0 machine — worth knowing before investing effort.
4	Wired ethernet, or a working USB/Thunderbolt adapter	24/7 duty on Wi-Fi is not acceptable for a box other people depend on.
5	Runs MemTest86 overnight without error	Salvaged RAM of unknown history is the most likely source of irreproducible failures on these machines.

Class A — Intel Mac laptop

Applies to Intel MacBook Pro models whose macOS support has ended. Both 2015 models cap at 16 GB soldered; 2016–2017 likewise. RAM is therefore not upgradeable and the tier is fixed by the chassis.

Capacity declaration			
CPU	Haswell/Broadwell mobile — i7-4870HQ class	Threads	4c/8t (13-inch: 2c/4t)
RAM	16 GB soldered, not upgradeable	Disk	NVMe via M.2 adapter, single slot
Concurrent seats	1 comfortable	Concurrent builds	1–2, thermally limited
Stacks	1 light	Binding constraint	Thermal, then CPU

A failing display is not a defect at this tier — the box is headless, so the broken part is the part never used. Confirm the fault is the display flex cable or panel and not backlight circuitry on the logic board, which is a different and terminal problem.

Why Ubuntu, not the last supported macOS

The machine’s final macOS stopped receiving security updates roughly two years ago, but obsolescence is not the deciding argument. Two others are:

- **Docker on macOS is a VM with a static memory carve-out.** The Linux VM’s RAM is allocated up front and cannot be rebalanced against host demand. On 16 GB there is no split that works: give the VM 8 GB and the host has 4,5 GB against a seat that peaks at 4,2 GB; give the VM less and the containers are starved. The whole premise of the capacity model above is that JVMs and containers peak at different moments and draw from one pool.
- **macOS costs roughly 3 GB to run.** WindowServer, Spotlight, and the desktop stack idle at ≈3,5 GB against Ubuntu Server’s 0,7 GB. On a 16 GB box that is a fifth of the machine spent on a GUI nobody looks at.

Practical consequences follow quickly: Homebrew provides no prebuilt bottles for macOS versions this old, so system tooling compiles from source on a decade-old mobile CPU, and current Docker Desktop will not install.

Against that, macOS keeps one genuine advantage — **battery charge limiting**. Optimized Battery Charging holds an always-plugged machine off 100 %, and Linux has no equivalent on Intel Macs because `applesmc` does not expose the charge threshold registers. Treat the battery as a consumable: inspect quarterly, replace when it swells. It is not a reason to keep macOS.

Commissioning runbook

Do this first, before wiping. Mac EFI firmware ships inside macOS updates. Install the last available macOS update, let it apply the final firmware, and only then wipe. Because no further macOS releases exist for this machine, no future firmware is lost by removing the partition — so there is no reason to keep a recovery partition or dual-boot.

1. **Battery.** Inspect before commissioning. A swollen cell in an always-on machine is a fire risk; the tells are a rising trackpad or a case that no longer sits flat. Do not “solve” this by removing the battery — Intel Macs throttle the CPU severely with no battery present, because the SMC will not let the adapter absorb turbo transients alone. A healthy battery is required, and it replaces the UPS line item entirely.
2. **Thermals.** Repaste the CPU and clean the fans. Original paste on a ten-year-old machine throttles under sustained load, and a throttling box produces exactly the timing variance the platform exists to design out.
3. **Storage.** The blade connector is proprietary; a Sintech-class M.2 adapter takes a standard NVMe drive and these models boot from it. One slot only — the two-disk split of Tier 4 is unavailable.

4. **Install media.** The current Ubuntu Server LTS — the playbook asserts properties, not a release. Hold Option at the chime and select EFI Boot. No T2 chip on this generation means no Secure Boot obstacle; these are among the easiest Macs to run Linux on.
5. **Attach wired ethernet during installation.** The Broadcom BCM43602 needs `bcmwl-kernel-source`, which cannot be downloaded without a network — a chicken-and-egg that strands the installer. A Thunderbolt or USB gigabit adapter sidesteps it and is required for 24/7 duty regardless.
6. **Declare the class in the box's inventory entry.** Everything after first boot is the Setup Guide's playbook, and it needs to know this is a laptop:

```
# inventory/host_vars/<hostname>.yaml (in the deployment)
dev_lid_ignore: true           # never suspend on lid close – it lives shut
dev_mbpfan: true              # the stock applesmc fan curve lets it cook
```

15-inch models with the discrete Radeon R9 M370X add `dev_kernel_module_blacklist: [radeon]` — the hybrid GPU is a known Linux problem and nothing here needs it. 2016–2017 chassis need the `applespi` driver for the internal keyboard and trackpad, mainline since kernel 5.3; it matters at the boot console and nowhere else.

7. **Burn-in.** MemTest86 overnight, then a day of the real E2E suite on loop with `bttop` watching package temperature. If it throttles below base clock under sustained load, the repaste did not take.

Class B — Ex-office desktop

Haswell/Broadwell-era small-form-factor and mini-tower machines — Dell OptiPlex, HP ProDesk/EliteDesk, Lenovo ThinkCentre. These are better dev boxes than their laptop contemporaries: the same microarchitecture at a higher sustained clock, in a chassis that does not throttle.

Capacity declaration

CPU	Haswell desktop — i5-4570 / i7-4770 class	Threads	4c/4t or 4c/8t
RAM	8 GB as found; board typically takes 32 GB	Disk	SATA SSD

Concurrent seats	1 sequenced
Stacks	1 minimal

Concurrent builds	2
Binding constraint	RAM — and it is cheap to fix

The binding constraint here is RAM, and unlike the laptop it is not soldered. That makes these machines Tier 2 candidates rather than permanent Tier 0, which is the single most important fact about them.

Commissioning runbook

1. **Count the DIMM slots before anything else.** Four-slot mini-tower and small-form-factor boards take 4×8 GB DDR3 for 32 GB. Two-slot ultra-small chassis cap at 16 GB with 8 GB modules — 16 GB unbuffered non-ECC DDR3 exists but is scarce and priced badly. Slot count decides which tier the machine can reach:

```
sudo dmidecode -t memory | grep -E "Number Of Devices|Maximum Capacity|Size:"
```

2. **Check the PSU and capacitors.** A decade-old office PSU driving 24/7 duty is the most likely hardware failure on these machines. Bulging capacitors on the board or PSU mean scrap, not repair.
3. **Clean thoroughly and repaste.** Tropical dust plus ten years of lab service.
4. **BIOS:** set **AC power loss** → **power on** so a headless box self-recovers when an outage outlasts nothing at all — these have no battery, unlike the laptop class, so a UPS is required if the power is unreliable.

5. **Disable everything unused:** onboard audio, serial and parallel ports, and the wireless card if one is fitted.
6. **Burn-in.** MemTest86 overnight on the RAM you intend to keep, including any newly bought modules.

Tier 1 — Compact

A 16 GB desktop carrying work that is not the Spring Boot + Postgres profile the capacity model is sized against: site maintenance, documentation, and feature work on interpreted stacks. Rootless Docker is installed and goes unused on most days, and the per-seat peak that drives every other tier's ceilings — a Maven build in flight beside a running application JVM — does not occur here.

That changes what the ceilings should be, not whether there are any. Sizing them against the capacity model's 5 GB seat — a Maven build beside an application JVM — would throttle legitimate work on this box. Sizing them against what a light seat actually holds gives a very different number:

Budget on a 16 GB box	MB
Physical	15.917
Ubuntu headless, plus one rootless daemon per active user	−1.900
Available to seats	14.017
Across five accounts	2.803 each

Hence **MemoryMax** of 2800M. Five seats peaking together reach 14.000 MB and leave the OS its 1.900 MB, so the box throttles and then kills the seat responsible rather than drifting into global OOM, where the kernel picks a victim unrelated to the cause. Keeping the failure attributable is the reason the ceilings exist.

Enforcement that fires in normal use is mis-sized. Light work does not come close to 2400M, and that is the intent — these numbers exist for the runaway case, not to ration ordinary work. If a seat starts meeting them routinely, the workload has changed and the answer is memory, not a larger number: two DIMM slots are free on this class of board, and 48 GB moves the box to Tier 2 where the ceilings are sized for JVM stacks.

What qualifies

Any machine that clears the Tier 0 triage gates and carries 16 GB. In practice that is an AM4 or LGA1151-era desktop bought for office use: four DIMM slots, a SATA SSD, a quad-core with SMT. The commissioning runbook is Tier 0's Class B — the hardware is the same class, only the memory fitted differs.

The registry pull-through mirror is off here. It earns its 0,2 GB by saving repeated image pulls, and a box that does not pull images repeatedly is spending resident memory to cache nothing.

Capacity declaration			
CPU	AM4 / LGA1151 desktop — Ryzen 5 1400 class	Threads	4c/8t
RAM	16 GB DDR4	Disk	SATA SSD, 240 GB or more
Concurrent seats	up to 5 light 2–3 comfortable	Concurrent builds	1–2
Stacks	1 light	Binding constraint	RAM

Two free DIMM slots make this the cheapest move to Tier 2 in the document: the same RAM-alone upgrade the tier below is built on, on newer silicon that has somewhere further to go afterwards.

Tier 2 — Salvage, upgraded

The same machines with their binding constraint removed. This is the best value in this document by a wide margin, and the lowest tier whose ceilings are sized for JVM work — Tier 1 is multiuser too, but only for seats lighter than the capacity model’s profile.

Upgrade	Est. IDR	Effect
4 × 8 GB DDR3-1600 (used, matched), per box	400.000 – 800.000	8 GB → 32 GB. One sequenced seat → 2–3 concurrent seats. Unlocks multiuser.
2 × 8 GB DDR3-1600 (used), 2-slot chassis	200.000 – 400.000	8 GB → 16 GB. Lands the machine in Tier 1 — one comfortable seat, or two on light work.
500 GB SATA SSD (new), if the machine failed gate 1	500.000	Clears the disqualifying gate. Mandatory, not optional.
1 TB SATA SSD (new), for per-user image stores	900.000	Required above two users — 25 GB of images each plus caches.

Capacity declaration			
CPU	Haswell desktop — i5-4570 / i7-4770 class	Threads	4c/4t or 4c/8t
RAM	32 GB DDR3	Disk	SATA SSD, 500 GB – 1 TB
Concurrent seats	2–3	Concurrent builds	2–3
Stacks	2 light	Binding constraint	CPU

Used DDR3 is near the bottom of its price curve and these boards are the last generation that takes it. Buy matched modules, run MemTest86 overnight on the assembled set, and treat any error as a reason to return the kit rather than diagnose it.

At 32 GB the binding constraint moves to the CPU, and the machine is finished — no further upgrade to a Haswell desktop is worth its cost. That is the signal to buy a Tier 3 box rather than keep investing here.

Two upgraded desktops beat one for a small team, because the CPU is the binding constraint and cores do not pool across a box boundary any worse than they pool inside one. Two 32 GB

boxes give 4–6 seats and survive one machine failing; a single 64 GB Haswell — were it possible — would give the same seats with none of the redundancy.

Tier 3 — Mid-range build

New hardware, bought when salvaged machines run out of seats and the primary rig is not yet justified. Prices are Tokopedia/Shopee-class street estimates — spot-check before ordering.

Component	Part	Est. IDR
CPU	AMD Ryzen 7 9700X (8c/16t, iGPU)	5.500.000
Cooler	Thermalright Peerless Assassin 120 SE	700.000
Motherboard	B650, 2.5GbE (MSI MAG / ASUS TUF class)	2.500.000
RAM	64 GB DDR5-5600 (2×32 GB)	3.500.000
Storage	1 TB NVMe Gen4	1.300.000
PSU	650 W 80+ Gold	1.200.000
Case	Mid-tower, sound-dampened, 2×140 mm	1.500.000
Total		≈ 16.200.000

Capacity declaration			
CPU	AMD Ryzen 7 9700X	Threads	8c/16t, homogeneous
RAM	64 GB DDR5	Disk	1 TB NVMe Gen4
Concurrent seats	5–6	Concurrent builds	6
Stacks	4–5 full	Binding constraint	CPU

Two DIMM slots left free take this to 128 GB later, at which point it becomes a Tier 4 box in all but core count. That upgrade path is the reason for 2×32 GB rather than 4×16 GB.

Tier 4 — Primary workstation

The tier at which **stack size** stops being a constraint: several full stacks at once — multi-broker Kafka, Oracle, monitoring, 10–12 GB each — with seats to spare. The workflow is the one every tier runs; what this box adds is that no project has to trim its stack to fit. Performance testing stays on rented cloud VMs; this box, like every other tier, is for functional correctness only.

Component	Part	Est. IDR
CPU	AMD Ryzen 9 9950X (16c/32t, iGPU — no discrete GPU)	10.000.000
Cooler	Thermalright Peerless Assassin 120 SE	700.000
Motherboard	MSI MAG B650 Tomahawk WiFi or ASUS TUF B650-Plus (2.5GbE, solid VRM)	4.000.000
RAM	128 GB DDR5-5600 (4×32 GB, e.g. Kingston Fury Beast)	7.000.000
Storage 1	2 TB NVMe Gen4 (WD SN850X / Kingston KC3000) — OS + repos	2.500.000

Component	Part	Est. IDR
Storage 2	2 TB NVMe Gen4 (same class) — Docker images/volumes + scratch	2.500.000
PSU	850 W 80+ Gold, semi-passive zero-RPM (Corsair RM850e / be quiet! Pure Power 12 M)	2.000.000
Case	Fractal Define 7 (sound-dampened, 3×140 mm fans included, external front filter)	3.800.000
UPS	1200 VA line-interactive with AVR (APC BX1200MI class)	2.300.000
Total		≈ 34.800.000

Capacity declaration			
CPU	AMD Ryzen 9 9950X	Threads	16c/32t, homogeneous
RAM	128 GB DDR5	Disk	2 × 2 TB NVMe Gen4, split OS/Docker
Concurrent seats	12+ (CPU-capped first)	Concurrent builds	9–12
Stacks	6–8 full	Binding constraint	Stack size, not seats

Remaining from a 50M envelope: ≈15M buffer. Upgrade paths if needed later: a third 2 TB NVMe when Docker volumes sprawl, UPS battery replacement at year two.

Deliberate omissions. No discrete GPU — the iGPU covers the rare BIOS or console session. No ECC RAM — DDR5 ECC UDIMMs are expensive and scarce locally; overnight MemTest86 plus conservative memory clocks covers the realistic risk for this use case.

Platform choice: why Ryzen, not Intel

The Intel alternatives at this price point were the Core i9-14900K (Raptor Lake) and the Core Ultra 9 285K (Arrow Lake). Both were rejected:

- **i9-14900K — reliability history.** That generation had a documented voltage-degradation defect: CPUs progressively destabilizing under sustained load, mitigated by microcode but with permanent damage on affected chips. Sustained all-core load, 24/7, for years is precisely the stress profile that surfaced the defect — and this box’s job is to be a trustworthy referee for correctness tests. No part with that history qualifies when a clean alternative exists at the same price.
- **Core Ultra 9 285K — heterogeneous cores.** The 285K mixes 8 P-cores with 16 E-cores; the 9950X has 16 identical full-performance cores. This workload is a dozen JVMs, stream-processing threads, and Testcontainers suites running concurrently — with hybrid cores, which core a thread lands on becomes a variable, and a test thread scheduled onto an E-core runs at a different speed than the same thread yesterday. That timing jitter manifests as exactly the flaky-test noise this machine exists to eliminate. Homogeneous cores remove one dimension of “same test, different result”.
- **Efficiency at the actual operating point.** Intel idles lower, but this box does not idle. Under sustained all-core load, Zen 5 capped at 105 W is the better perf/watt point, and eco mode is a one-setting BIOS feature.
- **Socket runway.** AM5 takes next-generation Ryzen as a drop-in upgrade on the same board; LGA1851 is expected to be a short-lived socket.

The 285K is not defective — it is stable and competitive in multi-core throughput; at a steep discount it would be a workable machine. At parity pricing, homogeneous cores, load efficiency, and platform longevity make the 9950X the straightforward pick.

Power & noise

At 24/7 saturation in eco mode: $\approx 160\text{--}180\text{ W}$ at the wall $\approx 1.500\text{ kWh/yr} \approx 2,6\text{ juta IDR/tahun}$ (PLN $\approx 1.700\text{ IDR/kWh}$). Peak $\approx 300\text{ W}$ is trivial even on a 2200 VA home connection. Noise is handled by the watt cap plus silent components — oversized dual-tower cooler idling at 400–600 RPM, 140 mm case fans at fixed low RPM, zero-RPM PSU, no GPU, NVMe only — landing at $\approx 20\text{--}25\text{ dBA}$ at 1 m. Placement outside the bedroom finishes the job; the box is headless over SSH anyway.

Commissioning runbook

1. **BIOS:** eco mode / cTDP 105 W ($\approx 85\text{--}90\%$ of stock multi-core throughput at $\approx 60\%$ power; single-thread nearly untouched); fan curves flat at minimum RPM until $70\text{ }^{\circ}\text{C}$; **AC power loss $\rightarrow$ power on** — a headless box must self-recover when an outage outlasts the UPS; EXPO off or capped at DDR5-4800 — four DIMMs at rated 5600 is unstable on AM5, and this machine's job is correctness testing, so memory stability beats bandwidth.
2. **Storage layout:** NVMe 1 (/) ext4 — OS and user homes. NVMe 2 mounted at /srv, ext4 — mirror data, loop account, and per-user Docker image stores, so overlay churn and volume I/O stay off the OS disk. Declare it in the box's inventory entry and the playbook points every user's daemon there:

```
# inventory/host_vars/<hostname>.yaml (in the deployment)
dev_docker_data_root: /srv/docker
```

3. **Burn-in before trusting it:** MemTest86 overnight (full 128 GB, all passes), then a day of the actual E2E suite on loop.
4. **Placement:** outside the bedroom, wired ethernet, front dust filter rinsed monthly (tropical dust + 24/7 duty).

Installing the operating system

Common to every class, and the point at which hardware becomes a box. The hardware runbooks above end at burn-in; the Setup Guide begins with a provisioning run as **root**. This is what closes the distance, and it is written out because two of its steps are things the installer will not do for you and will not warn you about.

1. **Write the install media.** Any current Ubuntu Server LTS — the playbook asserts properties, not a release string. Verify the download against the published **SHA256SUMS** before writing it; a corrupt image fails deep into the install, where the cause is not obvious.
2. **Set the hostname to the inventory name, during the install.** `preflight` asserts `inventory_hostname == ansible_facts.hostname`, because `ansible-pull` limits itself by hostname and a mismatch means the box silently never provisions itself. Renaming afterwards works but touches `/etc/hosts` too; deciding the name before you start is free.
3. **Guided storage, ext4, and take the whole disk.** `preflight` refuses to enable quotas on anything but ext4. Guided LVM leaves most of the volume group unallocated — check afterwards with `vgs` and `extend`, or the disk budget in this document is fiction.
4. **The account the installer creates is not a developer account.** Developers get theirs from the roster later. This one exists to bootstrap the box and then to be the console recovery account: it is the only login that will work at the physical keyboard once SSH is key-only, because every roster account has a locked password. Name it for that job rather than after a person, and record its password somewhere you will still have when SSH is what is broken.
5. **Import your SSH key in the installer.** `subiquity` offers to fetch a public key straight from a GitHub username. Take it — otherwise you are typing a key into a console by hand.

6. **After first boot, put the key on root.** This is the step that is missed, because the installer's key import puts it only on the account it created, and the Setup Guide connects as **root**:

```
sudo install -d -m 700 /root/.ssh
sudo install -m 600 ~/.ssh/authorized_keys /root/.ssh/authorized_keys
```

Verify from your own machine before handing over — `ssh root@<box>` must work without a password. A provisioning run that fails here fails at its first task, with an error about the connection rather than about the missing key.

Why root rather than sudo. The inventory connects as `ansible_user=root`, and developer accounts deliberately have no sudo — the isolation the platform provides rests on developers being unprivileged. Handing over a box whose only privileged access is a sudo user means either the playbook cannot run or that user has to become an exception. Root by key, everyone else unprivileged, is the shape the rest of the design assumes.

Choosing and combining tiers

Cost per seat

Tier	Build	Cost IDR	Seats	Per seat
0	Salvage, as found	0	1 seq.	0
1	16 GB desktop, as found	0	2–3 light	0
2	Salvage + 32 GB DDR3	±600.000	2–3	±240.000
3	Mid-range new build	±16.200.000	5–6	±2.900.000
4	Primary workstation	±34.800.000	6–8 stacks	±5.000.000

Tier 2 is an order of magnitude cheaper per seat than anything new. It is also capped: two upgraded desktops reach 4–6 seats and stop, because Haswell cores do not get faster and the boards take no more memory. The tiers are a ladder, not a menu — Tier 2 buys time, not a destination.

Staging path

1. **Now, at zero cost.** Salvaged machines at Tier 0 run the standing E2E loops, one project each. This removes mechanical verification from whatever machine developers actually work on, which is the highest-value thing these boxes do.
2. **At ±600.000 per box.** Upgrade the desktops to 32 GB. Each becomes a 2–3 seat multiuser box. For a small team this may be sufficient indefinitely, and it defers the primary rig purchase entirely.
3. **When seats run out.** Add Tier 3. One mid build roughly matches three upgraded salvage desktops in seats, in one chassis, with current-generation single-thread speed — which is what actually determines how long a build takes.
4. **When stacks run out.** Add Tier 4. The trigger is not seat count but stack size: the moment projects need full stacks with multi-broker Kafka, Oracle, and monitoring at 10–12 GB each, everything below Tier 4 runs out of memory regardless of how many boxes are stood up.

Mixed-tier fleets

Tiers combine freely because the platform is identical on all of them: same OS, same rootless model, same folder layout, same workflow, provisioned by the same playbook. A developer moves between boxes and finds the same environment; only the seat count differs.

Two rules govern placement:

- **Standing loops go to the lowest tier that can run them.** They are mechanical, unattended, and single-stack — exactly what a salvaged machine does well, and exactly the load worth keeping off a box where people are waiting on builds.
- **A project's stack lives on one box.** Splitting a stack across machines introduces network latency into test timing, which reintroduces the flakes the whole capacity model exists to prevent. Boxes divide work by project and by seat, never by service.